

Web Tech

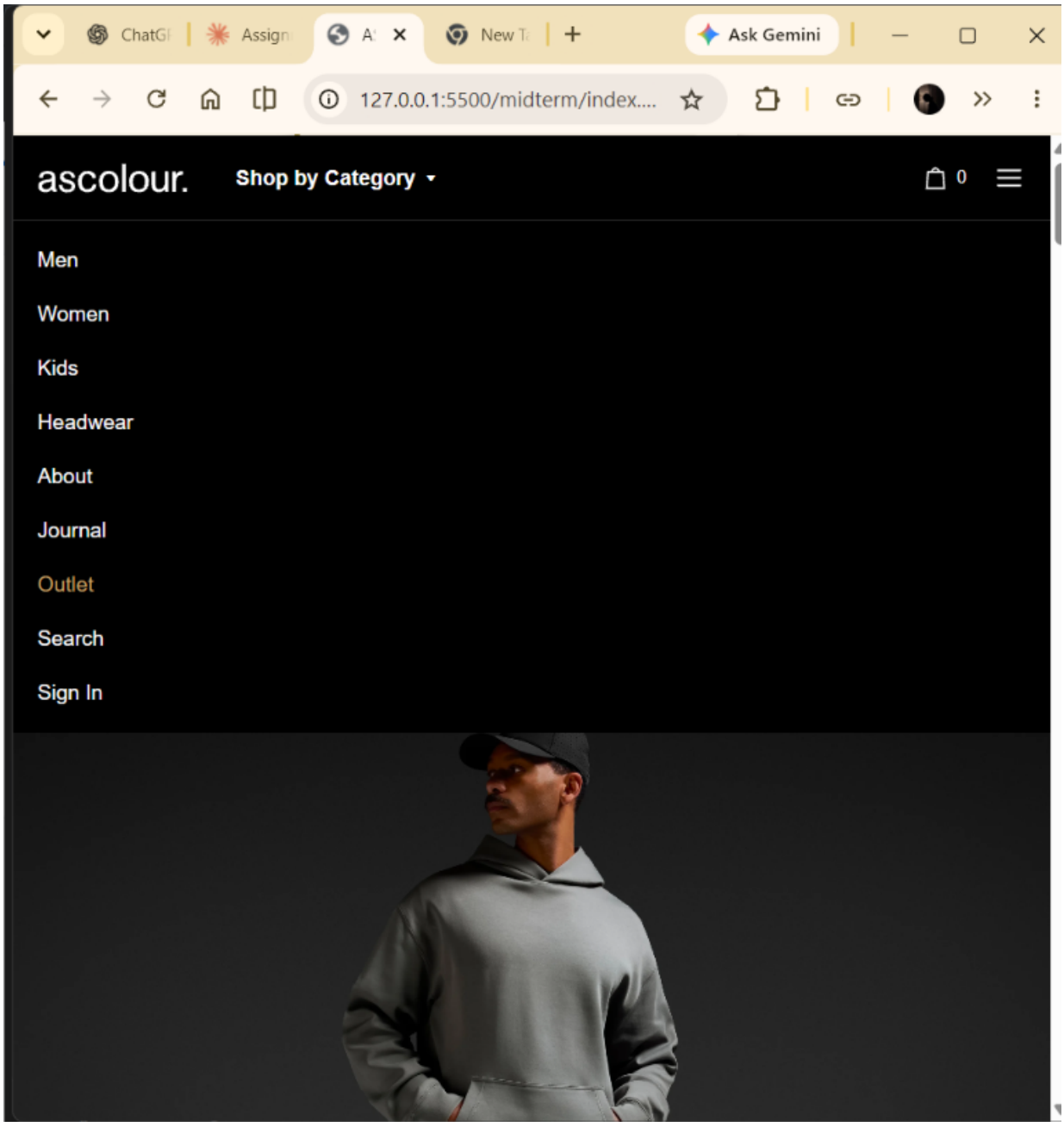
Mid term exam

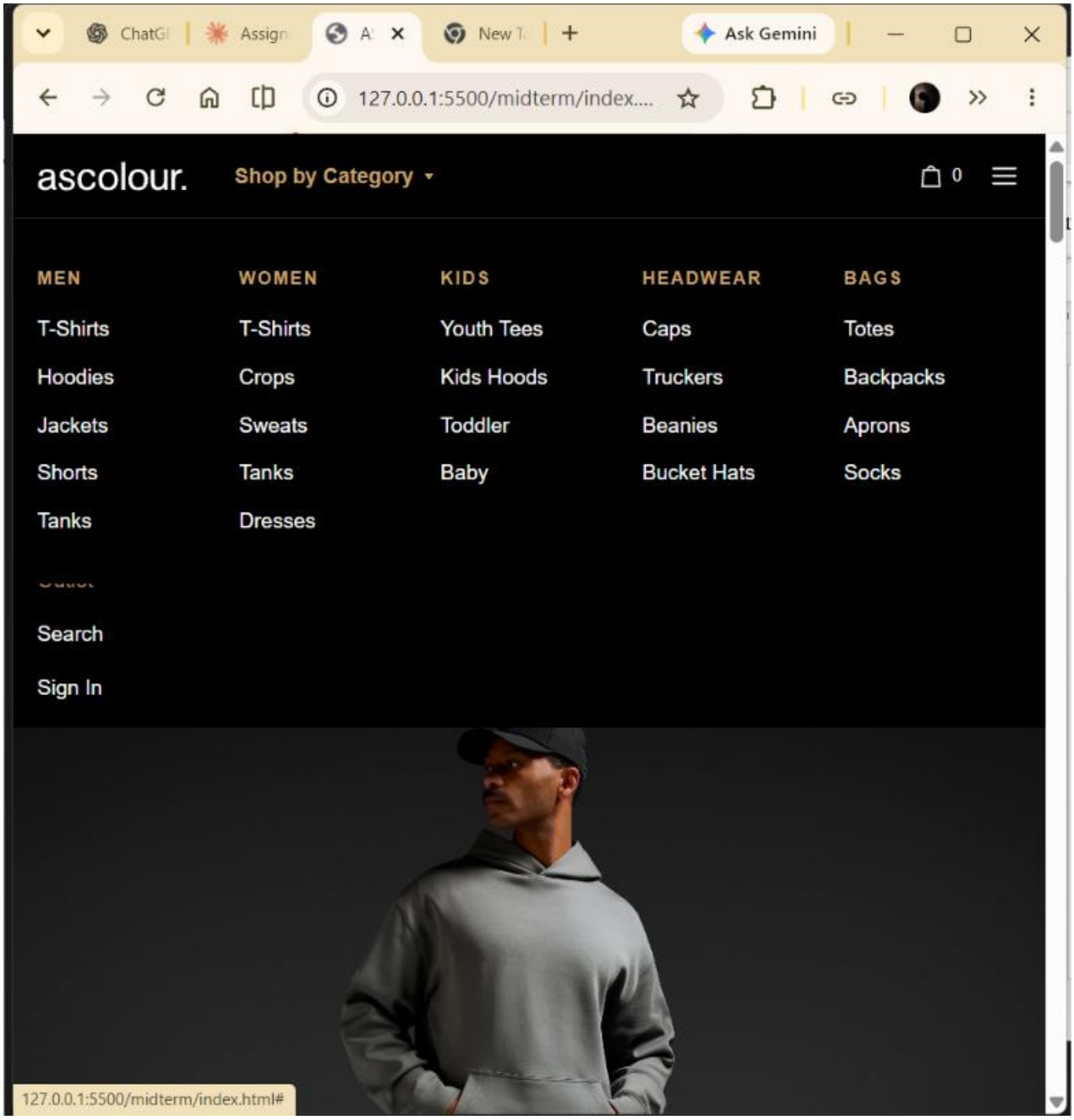
Submitted By: Amna Amjad

Registration no: FA24 – BCS - 017

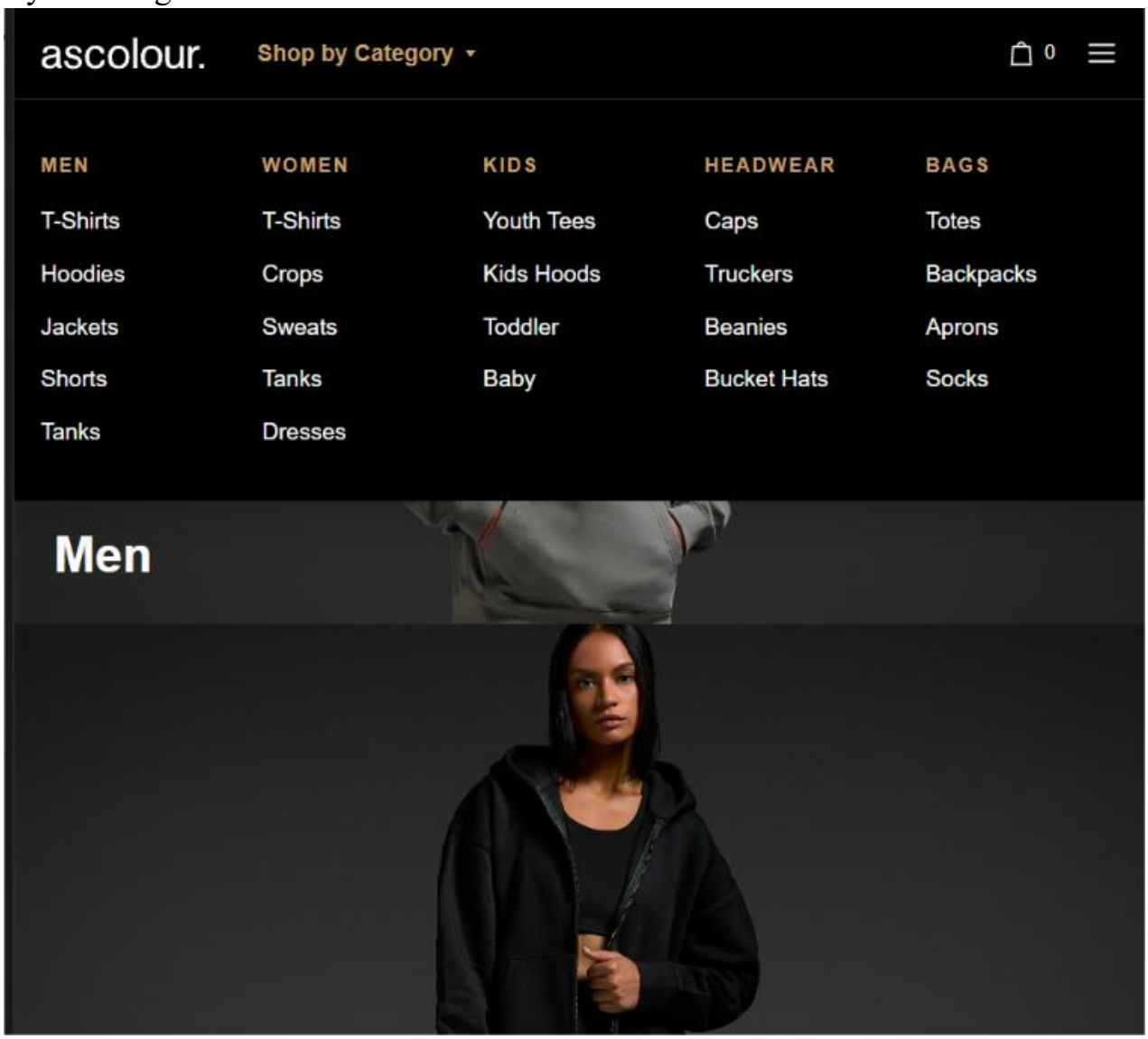
Task 1

My website in mobile format

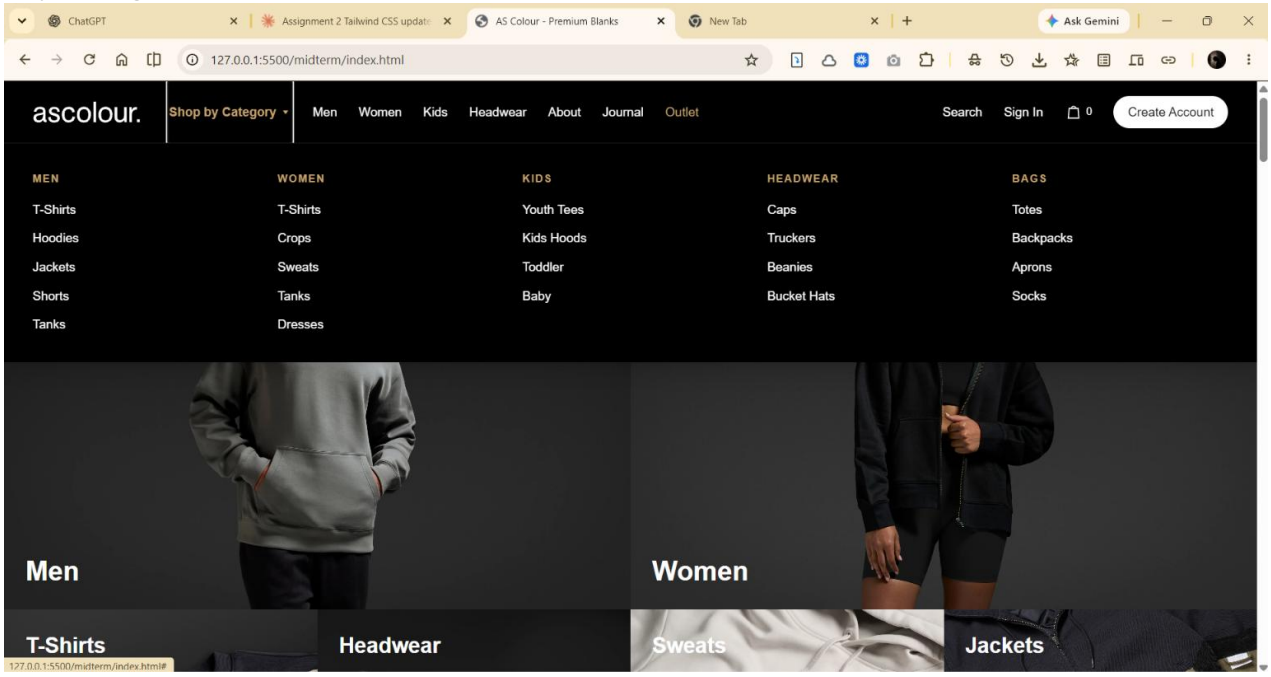



ChatGI
Assign
A
New T
Ask Gemini
127.0.0.1:5500/midterm/index....
ascolour.
Shop by Category
0
MEN
T-Shirts
Hoodies
Jackets
Shorts
Tanks
WOMEN
T-Shirts
Crops
Sweats
Tanks
Dresses
KIDS
Youth Tees
Kids Hoods
Toddler
Baby
HEADWEAR
Caps
Truckers
Beanies
Bucket Hats
BAGS
Totes
Backpacks
Aprons
Socks
Search
Sign In
127.0.0.1:5500/midterm/index.html#
,

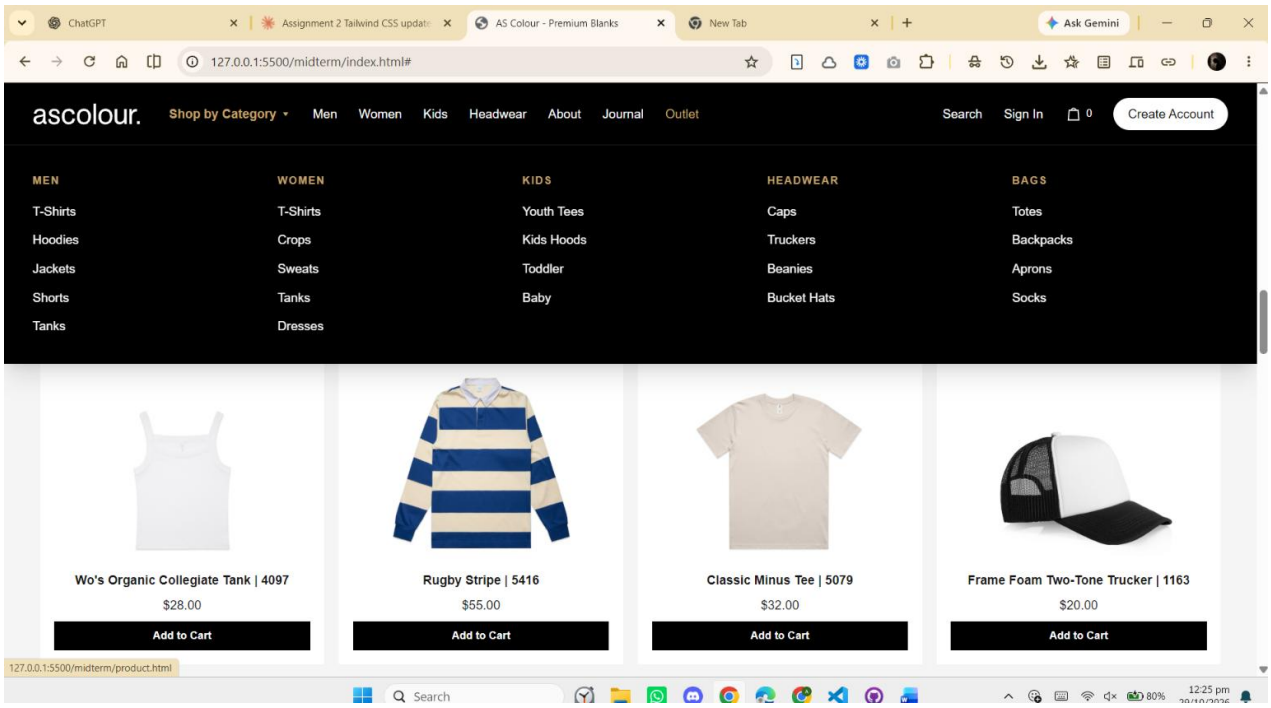
By hovering


ascolour.
Shop by Category
0
MEN
T-Shirts
Hoodies
Jackets
Shorts
Tanks
WOMEN
T-Shirts
Crops
Sweats
Tanks
Dresses
KIDS
Youth Tees
Kids Hoods
Toddler
Baby
HEADWEAR
Caps
Truckers
Beanies
Bucket Hats
BAGS
Totes
Backpacks
Aprons
Socks
Men

By using tab button



Menu over the product grid



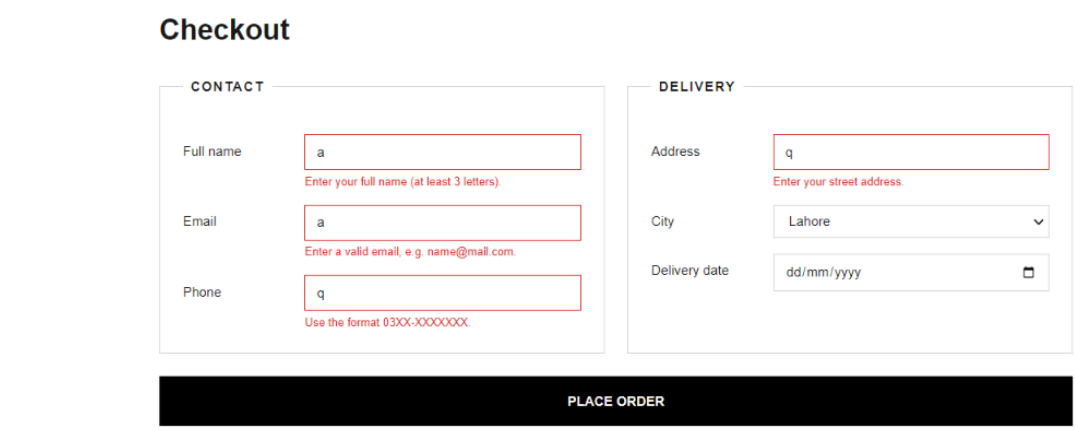
Code

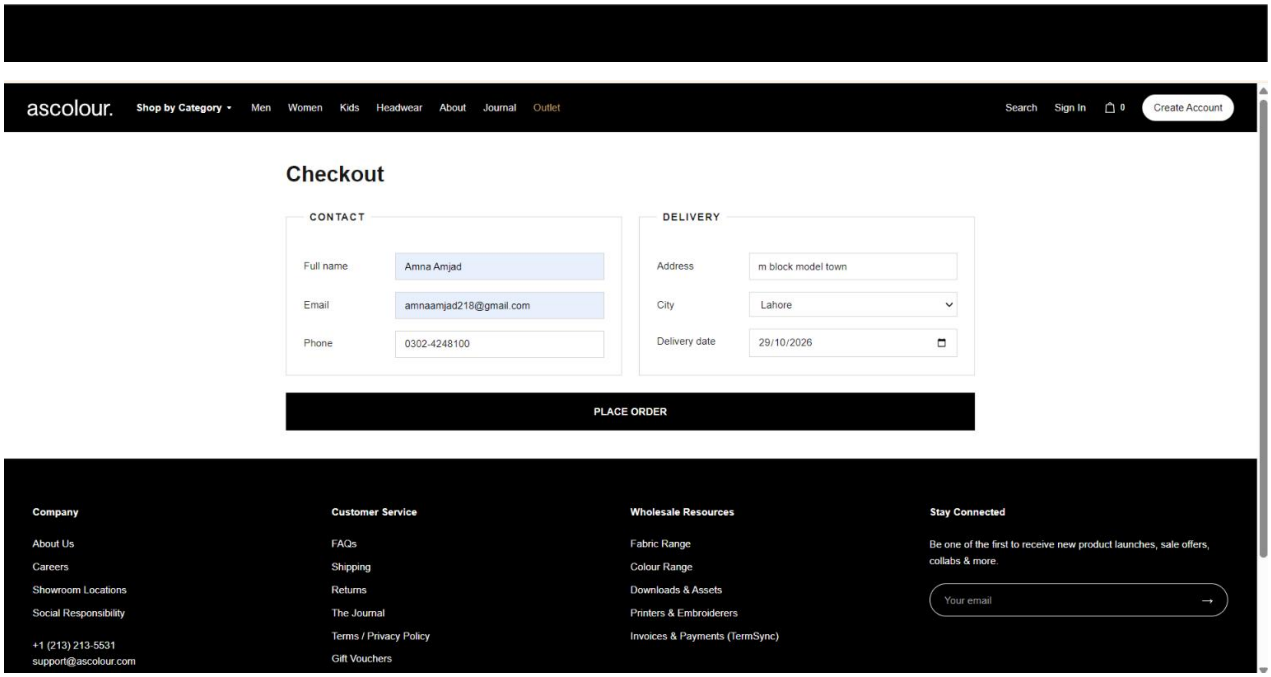
```
<!-- ===== TASK 1: Mobile accordion (below BP = 617px) =====
     The same 5 groups as the mega-menu, made with <details>/<summary>.
     Closed by default, each one opens on its own.
     "bp:hidden" hides it from 617px up, where the mega-menu takes over. -->
<div class="bp:hidden">
  <p class="mb-2 text-xs font-bold uppercase tracking-widest text-gold">Shop by Category</p>
  <details class="group border-b border-neutral-800">
    <summary class="flex cursor-pointer list-none justify-between py-3 after:content-['+'] group-open:after:content-['−']">Men</summary>
    <ul class="space-y-3 pb-4 pl-4">
      <li><a href="#">T-Shirts</a></li>
      <li><a href="#">Hoodies</a></li>
      <li><a href="#">Jackets</a></li>
      <li><a href="#">Shorts</a></li>
      <li><a href="#">Tanks</a></li>
    </ul>
  </details>
  <details class="group border-b border-neutral-800">
    <summary class="flex cursor-pointer list-none justify-between py-3 after:content-['+'] group-open:after:content-['−']">Women</summary>
    <ul class="space-y-3 pb-4 pl-4">
      <li><a href="#">T-Shirts</a></li>
      <li><a href="#">Crops</a></li>
      <li><a href="#">Sweats</a></li>
      <li><a href="#">Tanks</a></li>
      <li><a href="#">Dresses</a></li>
    </ul>
  </details>
```

```
            <details class="group border-b border-neutral-800">
                <summary class="flex cursor-pointer list-none justify-between py-3 after:content-['+'] group-open:after:content-['−']">Kids</summary>
                <ul class="space-y-3 pb-4 pl-4">
                    <li><a href="#">Youth Tees</a></li>
                    <li><a href="#">Kids Hoods</a></li>
                    <li><a href="#">Toddler</a></li>
                    <li><a href="#">Baby</a></li>
                </ul>
            </details>
            <details class="group border-b border-neutral-800">
                <summary class="flex cursor-pointer list-none justify-between py-3 after:content-['+'] group-open:after:content-['−']">Headwear</summary>
                <ul class="space-y-3 pb-4 pl-4">
                    <li><a href="#">Caps</a></li>
                    <li><a href="#">Truckers</a></li>
                    <li><a href="#">Beanies</a></li>
                    <li><a href="#">Bucket Hats</a></li>
                </ul>
            </details>
            <details class="group border-b border-neutral-800">
                <summary class="flex cursor-pointer list-none justify-between py-3 after:content-['+'] group-open:after:content-['−']">Bags</summary>
                <ul class="space-y-3 pb-4 pl-4">
                    <li><a href="#">Totes</a></li>
                    <li><a href="#">Backpacks</a></li>
                    <li><a href="#">Aprons</a></li>
                    <li><a href="#">Socks</a></li>
                </ul>
            </details>
        </div>
```

<!-- ===== end TASK 1 mobile accordion ===== -->

Task 2





Code

TASK 2 - Checkout form with HTML5 validation only

==========

1) midterm/checkout.html (NEW FILE) - <head> part

<title>Checkout - AS Colour</title>

```
  <!-- TASK 2: same Tailwind CSS file as the rest of the site (midterm/css/output.css) -->
  <link rel="stylesheet" href="css/output.css">
```

2) midterm/checkout.html (NEW FILE) - checkout form

(The page also reuses the same header and footer as index.html.)

```
  <!-- ===== TASK 2: Checkout form (HTML5 validation + CSS only, no JavaScript) ===== -->
  <main class="mx-auto max-w-5xl px-4 py-10 md:px-8">
    <h1 class="mb-8 text-2xl font-bold md:text-3xl">Checkout</h1>

    <!-- TASK 2: the browser checks required, type, pattern, min and max before submitting.
         Layout: 1 column on phone; from BP (617px) the two fieldsets sit side by side. -->
    <form action="#" method="post" class="grid gap-6 bp:grid-cols-2">

      <!-- ===== Fieldset 1: Contact ===== -->
      <!-- min-w-0 stops a fieldset from overflowing its grid column -->
      <fieldset class="min-w-0 space-y-4 border border-neutral-300 p-4 md:p-6">
        <legend class="px-2 text-sm font-bold uppercase tracking-widest">Contact</legend>

        <!-- TASK 2: each row = label + input.
             Below BP: label above input.
             From BP: grid with a fixed label column (7rem) on the left, input on the right (minmax(0,1fr) stops inputs overflowing). -->
        <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
```

```
            <label for="fullname" class="text-sm bp:pt-2">Full name</label>
            <div>
              <!-- "peer" lets the message below react to this input's state.
                   [&:user-invalid] = red border only AFTER the user has interacted. -->
              <input id="fullname" name="fullname" type="text" required minlength="3" autocomplete="name"
                     class="peer w-full border border-neutral-300 px-3 py-2 text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
              <p class="mt-1 hidden text-xs text-red-600 peer-[:user-invalid]:block">Enter your full name (at least 3 letters).</p>
            </div>
          </div>

          <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
            <label for="email" class="text-sm bp:pt-2">Email</label>
            <div>
              <input id="email" name="email" type="email" required autocomplete="email"
                     class="peer w-full border border-neutral-300 px-3 py-2 text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
              <p class="mt-1 hidden text-xs text-red-600 peer-[:user-invalid]:block">Enter a valid email, e.g. name@mail.com.</p>
            </div>
          </div>

          <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
            <label for="phone" class="text-sm bp:pt-2">Phone</label>
            <div>
```

```
                <!-- TASK 2: pattern = 03 + 2 digits + dash + 7 digits (03XX-XXXXXXX) -->
                <input id="phone" name="phone" type="tel" required pattern="03[0-9]{2}-[0-9]{7}" placeholder="03XX-XXXXXXX" autocomplete="tel"
                       class="peer w-full border border-neutral-300 px-3 py-2 text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
                <p class="mt-1 hidden text-xs text-red-600 peer-[:user-invalid]:block">Use the format 03XX-XXXXXXX.</p>
              </div>
            </div>
          </fieldset>

          <!-- ===== Fieldset 2: Delivery ===== -->
          <fieldset class="min-w-0 space-y-4 border border-neutral-300 p-4 md:p-6">
            <legend class="px-2 text-sm font-bold uppercase tracking-widest">Delivery</legend>

            <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
              <label for="address" class="text-sm bp:pt-2">Address</label>
              <div>
                <input id="address" name="address" type="text" required minlength="5" autocomplete="street-address"
                       class="peer w-full border border-neutral-300 px-3 py-2 text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
                <p class="mt-1 hidden text-xs text-red-600 peer-[:user-invalid]:block">Enter your street address.</p>
              </div>
            </div>
```

```
            <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
                <label for="city" class="text-sm bp:pt-2">City</label>
                <div>
                    <!-- TASK 2: city as a <select>; the empty first option makes "required" work -->
                    <select id="city" name="city" required
                        class="peer w-full border border-neutral-300 bg-white px-3 py-2 text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
                        <option value="">Choose a city</option>
                        <option>Lahore</option>
                        <option>Karachi</option>
                        <option>Islamabad</option>
                        <option>Rawalpindi</option>
                        <option>Faisalabad</option>
                        <option>Multan</option>
                        <option>Peshawar</option>
                    </select>
                    <p class="mt-1 hidden text-xs text-red-600 peer-[:user-invalid]:block">Choose a city.</p>
                </div>
            </div>

            <div class="grid gap-1 bp:grid-cols-[7rem_minmax(0,1fr)] bp:gap-x-4">
                <label for="date" class="text-sm bp:pt-2">Delivery date</label>
                <div>
                    <!-- TASK 2: min = exam date (29 Oct 2026), max = exam date + N (17) days = 15 Nov 2026 -->
                    <input id="date" name="date" type="date" required
                        min="2026-10-29" max="2026-11-15"
```

```
                    class="peer w-full border border-neutral-300 px-3 py-2
text-sm outline-none focus:border-black [&:user-invalid]:border-red-600">
                <p class="mt-1 hidden text-xs text-red-600 peer-[:user-
invalid]:block">Pick a date from 29 Oct to 15 Nov 2026.</p>
              </div>
            </div>
          </fieldset>

          <!-- TASK 2: submit button is full width at every size (spans both
columns from BP) -->
          <button type="submit" class="w-full bg-black py-4 text-sm font-bold
uppercase text-white hover:bg-neutral-700 bp:col-span-2">Place
Order</button>
        </form>
      </main>
      <!-- ===== end TASK 2 ===== -->
```

3) midterm/index.html and midterm/product.html - cart icon link (CHANGED)

Before:

```
          <a href="#" aria-label="Cart" class="flex items-center gap-1">
```

After:

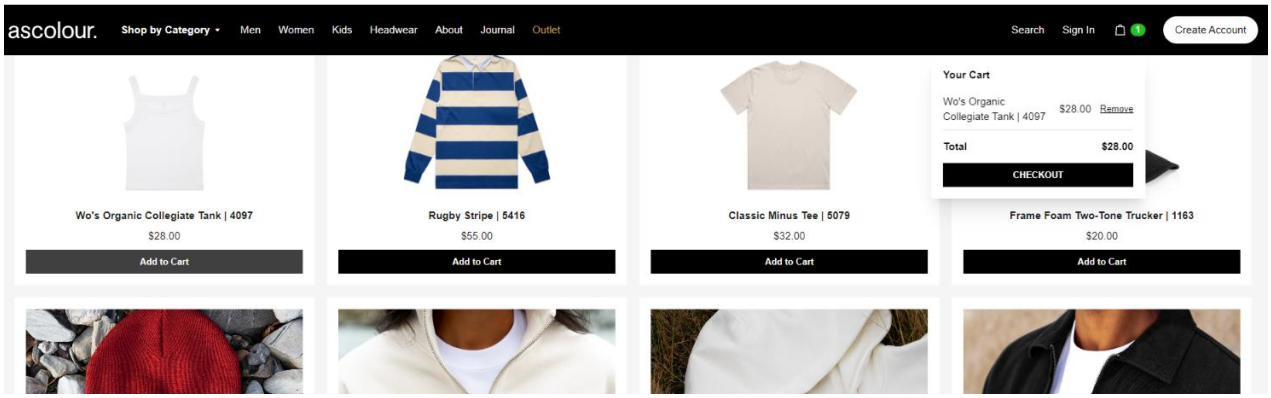
```
          <!-- TASK 2: cart icon now opens the checkout page -->
          <a href="checkout.html" aria-label="Cart" class="flex items-center
gap-1">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" stroke-
width="1.5" viewBox="0 0 24 24" aria-hidden="true"><path d="M6 7h12l1
13H5L6 7z"/><path d="M9 7a3 3 0 0 1 6 0"/></svg>
            <span class="text-xs">0</span>
          </a>
```

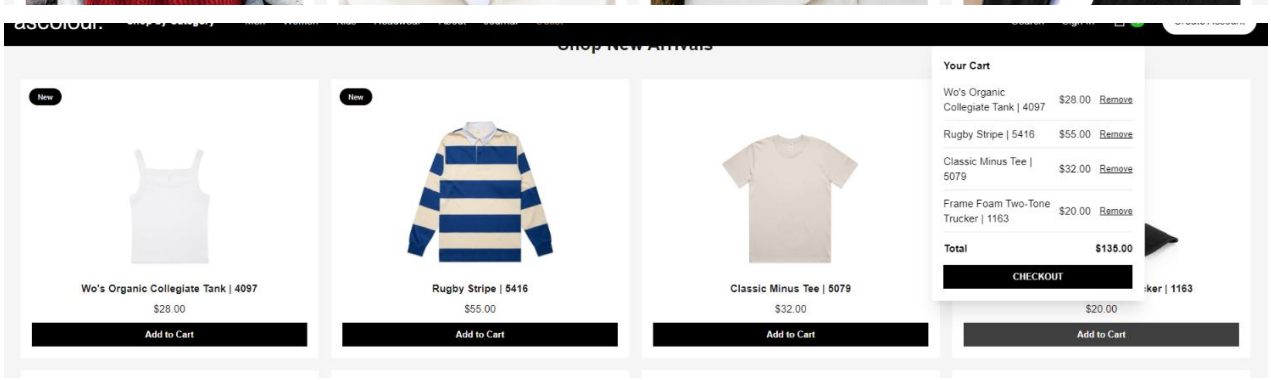
4) midterm/tailwind.config.js (CHANGED)

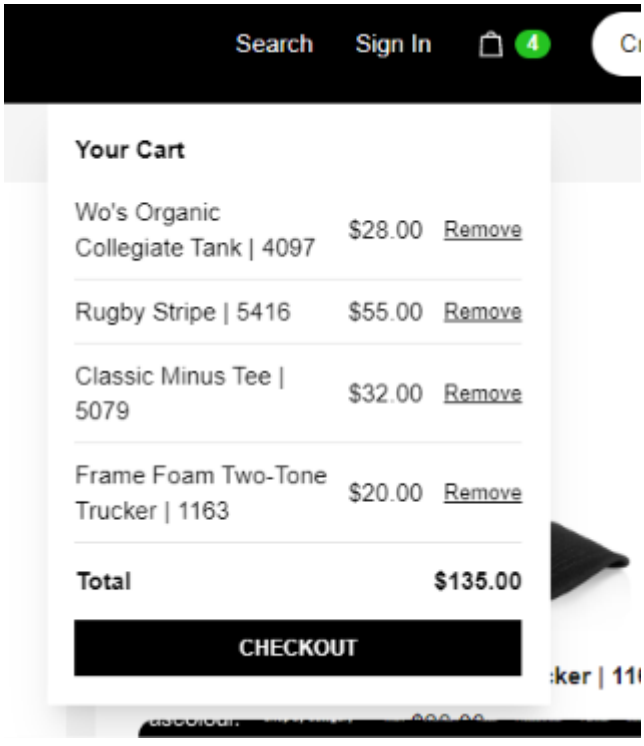
// TASK 2: also scan the new checkout page

content: ['./index.html', './product.html', './checkout.html'],

now **task 3**







Task 3 code

TASK 3 - JavaScript: working cart badge and mini cart

==========

1) midterm/script.js (NEW FILE)

```
// ===== TASK 3: Cart badge + mini cart (plain JavaScript) =====
// My numbers: N = 17, so each product can be added at most (17 % 4) + 2 = 3 times.
const MAX_PER_PRODUCT = (17 % 4) + 2; // = 3

// The cart: an array of { name, price } objects
let cart = [];

// Elements already on the page
const addButtons = document.querySelectorAll('.add-btn');   // the 8 "Add to Cart" buttons
const cartIcon   = document.getElementById('cart-icon');     // cart icon in the navbar
const badge      = document.getElementById('cart-count');    // number badge on the icon
const miniCart   = document.getElementById('mini-cart');     // the small panel
const itemsList  = document.getElementById('cart-items');    // <ul> inside the panel
const emptyText  = document.getElementById('cart-empty');    // "Your cart is empty"
const totalText  = document.getElementById('cart-total');    // total price

// How many times a product is already in the cart
function countOf(name) {
    return cart.filter(item => item.name === name).length;
```

```
}

// 1) "Add to Cart": read the name and price from the card itself
addButtons.forEach(button => {
    button.addEventListener('click', () => {
        const card  = button.closest('.product-card');
        const name  = card.querySelector('h3').textContent.trim();
        const price =
parseFloat(card.querySelector('.price').textContent.replace('$', ''));

        if (countOf(name) < MAX_PER_PRODUCT) {
            cart.push({ name: name, price: price });
            updateCart();
        }
    });
});

// 2) Cart icon: open / close the mini cart (instead of going to checkout)
cartIcon.addEventListener('click', (event) => {
    event.preventDefault(); // stop the link; the panel has its own "Checkout"
link
    miniCart.classList.toggle('hidden');
    cartIcon.setAttribute('aria-expanded', !miniCart.classList.contains('hidden'));
});

// 3) "Remove" buttons inside the panel (one listener on the list)
itemsList.addEventListener('click', (event) => {
    if (event.target.classList.contains('remove-btn')) {
        const index = Number(event.target.dataset.index);
        cart.splice(index, 1); // remove that one item from the array
        updateCart();
```

```
    }
});

// Redraw everything: list, total, badge and button states
function updateCart() {
    // List: one row per item, each with its price and a Remove button
    itemsList.innerHTML = '';
    cart.forEach((item, index) => {
        itemsList.innerHTML += `
            <li class="flex items-center justify-between gap-3 border-b border-neutral-200 py-2">
                <span class="flex-1">${item.name}</span>
                <span>$${item.price.toFixed(2)}</span>
                <button type="button" class="remove-btn text-xs underline" data-index="${index}">Remove</button>
            </li>`;
    });
    emptyText.classList.toggle('hidden', cart.length > 0);

    // Total with reduce()
    const total = cart.reduce((sum, item) => sum + item.price, 0);
    totalText.textContent = '$' + total.toFixed(2);

    // Badge: number of items (hidden when the cart is empty)
    badge.textContent = cart.length;
    badge.classList.toggle('hidden', cart.length === 0);

    // Limit: disable a product's button once it is in the cart 3 times
    addButtons.forEach(button => {
        const name = button.closest('.product-card').querySelector('h3').textContent.trim();
```

```
      const full = countOf(name) >= MAX_PER_PRODUCT;
      button.disabled = full;
      button.textContent = full ? 'Limit reached' : 'Add to Cart';
    });
}

updateCart();
// ===== end TASK 3 =====
```

2) midterm/index.html - cart icon, badge and mini-cart panel (CHANGED)

```
            <!-- TASK 3: wrapper is "relative" so the mini cart opens right under the icon -->
            <div class="relative flex items-center">
              <!-- TASK 2: cart icon links to the checkout page (works without JavaScript).
                   TASK 3: script.js stops the link and opens the mini cart instead. -->
              <a href="checkout.html" id="cart-icon" aria-label="Cart" aria-controls="mini-cart" aria-expanded="false" class="flex items-center gap-1">
                <svg class="h-5 w-5" fill="none" stroke="currentColor" stroke-width="1.5" viewBox="0 0 24 24" aria-hidden="true"><path d="M6 7h12l1 13H5L6 7z"/><path d="M9 7a3 3 0 0 1 6 0"/></svg>
                <!-- TASK 3: badge with the number of items (accent colour hsl(119, 70%, 45%)) -->
                <span id="cart-count" class="hidden min-w-[1.25rem] rounded-full bg-[hsl(119,70%,45%)] px-1.5 text-center text-xs font-bold text-white">0</span>
              </a>

              <!-- ===== TASK 3: Mini cart panel (hidden until the cart icon is clicked) ===== -->
```

```
            <div id="mini-cart" class="absolute right-0 top-full z-50 mt-4 hidden w-72 lg:mt-6 max-w-[calc(100vw-2rem)] bg-white p-4 text-sm text-black shadow-xl">
                <h2 class="mb-2 font-bold">Your Cart</h2>
                <p id="cart-empty" class="py-2 text-neutral-500">Your cart is empty.</p>
                <ul id="cart-items"></ul>
                <div class="mt-3 flex justify-between font-bold">
                  <span>Total</span>
                  <span id="cart-total">$0.00</span>
                </div>
                <a href="checkout.html" class="mt-3 block bg-black py-2 text-center text-xs font-bold uppercase text-white hover:bg-neutral-700">Checkout</a>
            </div>
            <!-- ===== end TASK 3 mini cart ===== -->
          </div>
```

3) midterm/index.html - product card hooks (CHANGED, same on all 8 cards)

```
  <!-- ===== Product Listing: 8 cards (Tailwind flex-wrap grid) =====
       Every card links to the same product page.
       TASK 3: hooks for script.js -> "product-card" (the card), "price" (its price),
       "add-btn" (its Add to Cart button). The disabled: classes grey out "Limit reached". -->
        <div class="mb-4 w-1/2 px-2 md:w-1/3 lg:w-1/4">
          <div class="product-card relative flex h-full flex-col bg-white p-4 text-center">
            <span class="absolute left-3 top-3 rounded-full bg-black px-3 py-1 text-[10px] text-white">New</span>
            <a href="product.html" class="flex flex-1 flex-col">
```

<img src="images/ascolour_030.jpg" alt="Wo's Organic Collegiate Tank" class="mb-4 h-40 w-full object-contain md:h-60">

<h3 class="text-xs font-bold md:text-sm">Wo's Organic Collegiate Tank | 4097</h3>

$28.00

<button type="button" class="add-btn w-full bg-black py-2 text-xs font-bold text-white hover:bg-neutral-700 disabled:cursor-not-allowed disabled:bg-neutral-400">Add to Cart</button>

4) midterm/index.html - script tag before </body> (ADDED)

<!-- TASK 3: cart script (plain JavaScript) -->

<script src="script.js"></script>

5) midterm/tailwind.config.js (CHANGED)

// TASK 2: also scan the new checkout page. TASK 3: and script.js (it creates the cart rows)

content: ['./index.html', './product.html', './checkout.html', './script.js'],